

assignment1

Assignment #1: OOP

Updated 2026-03-10 11:25 GMT+8
Compiled by 徐前 物理学院 (2026 Spring)

作业的各项评分细则及对应的得分

标准	等级	得分
按时提交	完全按时提交：1分 提交有请假说明：0.5分 未提交：0分	1 分
源码、耗时（可选）、解题思路（可选）	提交了4个或更多题目且包含所有必要信息：1分 提交了2个或以上题目但不足4个：0.5分 少于2个：0分	1 分
AC代码截图	提交了4个或更多题目且包含所有必要信息：1分 提交了2个或以上题目但不足4个：0.5分 少于：0分	1 分
清晰头像、PDF文件、MD/DOC附件	包含清晰的Canvas头像、PDF文件以及MD或DOC格式的附件：1分 缺少上述三项中的任意一项：0.5分 缺失两项或以上：0分	1 分
学习总结和个人收获	提交了学习总结和个人收获：1分 未提交学习总结或内容不详：0分	1 分
总得分： 5	总分满分：5分	

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge， Codeforces， LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. 课程平台：

课程网站位于Canvas平台 (<https://pku.instructure.com>)。该平台将在第2周选课结束后正式启用。在平台启用前，请先完成作业并将作业妥善保存。待Canvas平台激活后，再上传你的作业。

3. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。
4. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E27653: Fraction类

OOP, <http://cs101.openjudge.cn/pctbook/E27653/>

主要是练习面向对象编程写法，这样力扣题目，笔试都没有问题了。机考时候，不是必须OOP，能AC就可以。

思路：

需要定义“分数”类，支持加法操作 `__add__`，如果输出需要重写 `__str__` 方法。

`__str__` 方法可以被 `print()` 函数、字符串格式化操作以及 `str()` 函数自动调用。

如何化成最简分数，可以将分子分母同时除以最大公约数 `gcd`。

代码：

```
class Fraction:
    def __init__(self, n, d):
        self.n = n
        self.d = d
    def __add__(self, otherf):
        newnum = self.n * otherf.d + self.d * otherf.n
        newden = self.d * otherf.d
        common = gcd(newnum, newden)
        return Fraction(newnum // common, newden // common)
    def __str__(self):
        return f'{self.n}/{self.d}'

def gcd(m, n):
    while m % n != 0:
        m, n = n, m % n
    return n

def main():
    line = input().split()
    n1 = int(line[0])
    d1 = int(line[1])
```

```
n2 = int(line[2])
d2 = int(line[3])

f1 = Fraction(n1, d1)
f2 = Fraction(n2, d2)

result = f1 + f2

print(result)

if __name__ == '__main__':
    main()
```

代码运行截图 (至少包含有"Accepted")

#51853958提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
def gcd(m, n):
    while m % n != 0:
        m, n = n, m % n
    return n
class Fraction:
    def __init__(self, n, d):
        self.n = n
        self.d = d
    def __add__(self, otherf):
        newnum = self.n * otherf.d + self.d * otherf.n
        newden = self.d * otherf.d
        common = gcd(newnum, newden)
        return Fraction(newnum // common, newden // common)
    def __str__(self):
        return f'{self.n}/{self.d}'
def main():
    line = input().split()
    n1 = int(line[0])
    d1 = int(line[1])
    n2 = int(line[2])
    d2 = int(line[3])

    f1 = Fraction(n1, d1)
    f2 = Fraction(n2, d2)
```

基本信息

#: 51853958
题目: E27653
提交人: 25n2500011619
内存: 3660kB
时间: 26ms
语言: Python3
提交时间: 2026-03-03 16:06:43

E190.颠倒二进制位

bit manipulation, <https://leetcode.cn/problems/reverse-bits/>

思路:

暴力方法是将n的二进制位逐一反转;

改进方法是采用分治的思想, 先交换前16位和后16位, 再反转各自的一半;

依次交换左右的8、4、2、1位, 时间复杂度为 $O(\log 32) = O(5)$

代码:

方法一 暴力

```
def reverseBits(self, n: int) -> int:
    ans = 0
    for i in range(32):
        ans = ans << 1 | n & 1
        n >>= 1
    return ans
```

方法二 分治

```
def reverseBits(self, n: int) -> int:
    # 步骤1: 交换每1位中的左右16位
    n = (n >> 16) | (n << 16)
    # 步骤2: 交换每8位中的左右8位
    n = ((n & 0xff00ff00) >> 8) | ((n & 0x00ff00ff) << 8)
    # 步骤3: 交换每4位中的左右4位
    n = ((n & 0xf0f0f0f0) >> 4) | ((n & 0x0f0f0f0f) << 4)
    # 步骤4: 交换每2位中的左右2位
    n = ((n & 0xcccccccc) >> 2) | ((n & 0x33333333) << 2)
    # 步骤5: 交换每1位中的左右1位
    n = ((n & 0xaaaaaaaa) >> 1) | ((n & 0x55555555) << 1)
    return n & 0xffffffff # 确保结果为32位
```

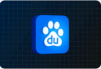
代码运行截图（至少包含有"Accepted"）

通过 600 / 600 个通过的测试用例

Jolly Turing00u 提交于 2026.03.03 17:25

官方题解

写题解



百度面试突击手册
众里寻你签百度



执行用时分布

51 ms | 击败 49.26%

复杂度分析

消耗内存分布

18.93 MB | 击败 67.33%



```
1 class Solution:
2     def reverseBits(self, n: int) -> int:
3         n = (n >> 16) | (n << 16)
4         n = (n & 0xff00ff00) >> 8 | (n & 0x00ff00ff) << 8
5         n = (n & 0xf0f0f0f0) >> 4 | (n & 0x0f0f0f0f) << 4
6         n = (n & 0xcccccccc) >> 2 | (n & 0x33333333) << 2
7         n = (n & 0xaaaaaaaa) >> 1 | (n & 0x55555555) << 1
8         return n & 0xffffffff
```

已存储

测试用例 | 测试结果

通过 执行用时: 63 ms

Case 1 Case 2

输入

n =
2147483644

输出

1073741822

E1356.根据数字二进制下 1 的数目排序

bit manipulation, <https://leetcode.cn/problems/sort-integers-by-the-number-of-1-bits/>

思路:

使用 `x.bit_count()` 统计 1 的个数, 使用 `sort()` 函数, 关键词key设为元组 `(x.bit_count, x)`

代码：

```
def sortByBits(self, arr: List[int]) -> List[int]:
    arr.sort(key = lambda x: (x.bit_count(), x))
    return arr
```

代码运行截图（至少包含有"Accepted"）



M27300: 模型整理

sortings, AI, <http://cs101.openjudge.cn/pctbook/M27300/>

思路：

用字典的键存储模型名，值存储参数量（包括数字和单位），然后排序key，分别排序value。
需要重新定义小于号 `__lt__`

代码：

```
from collections import defaultdict

class llm:
    def __init__(self, string: str) -> None:
        self.m = string
        self.unit = string[-1]
        self.num = float(string[:-1])

    def __lt__(self, other):
```

```

        if self.unit == other.unit:
            return self.num < other.num
        return self.unit == 'M'

```

```

def main():
    dic = defaultdict(list)
    n = int(input())
    for _ in range(n):
        name, m = input().split('-')
        dic[name].append(llm(m))
    names = sorted(dic.keys())
    for name in names:
        dic[name].sort()
        print(f"{name}: {' , '.join([j.m for j in dic[name]])}")

```

```

if __name__ == '__main__':
    main()

```

定义一个 ParamSize 类，继承 str ，专门用于定义模型大小的排序。

```

class Paramsize(str):
    def __lt__(self, other):
        unita = self[-1]
        unitb = other[-1]
        if unita == unitb:
            numa = float(self[:-1])
            numb = float(other[:-1])
            return numa < numb
        return unita == 'M'

```

```

def main():
    dic = defaultdict(list)
    n = int(input())
    for _ in range(n):
        name, m = input().split('-')
        dic[name].append(Paramsize(m))
    names = sorted(dic.keys())
    for name in names:
        dic[name].sort()
        print(f"{name}: {' , '.join(dic[name])}")

```

```

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

#51855024提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
from collections import defaultdict
class llm:
    def __init__(self, string:str) -> None:
        self.name, self.m = string.split('-')
        self.unit = self.m[-1]
        self.num = float(self.m[:-1])
    def __lt__(self, other):
        if self.unit == other.unit:
            return self.num < other.num
        return self.unit == 'M'

def main():
    dic = defaultdict(list)
    n = int(input())
    for _ in range(n):
        mode = llm(input())
        dic[mode.name].append(mode)
```

基本信息

#: 51855024
题目: M27300
提交人: 25n2500011619
内存: 3664kB
时间: 24ms
语言: Python3
提交时间: 2026-03-03 18:17:27

M1536.排布二进制网格的最少交换次数

greedy, matrix, <https://leetcode.cn/problems/minimum-swaps-to-arrange-a-binary-grid/>

思路:

预处理第 i 行最后一个 1 的位置 $pos[i]$, 对于从上到下的每一行, 找到 $j \geq i$ 中第一个 $pos[j] \leq i$ 的第 j 行, 将其冒泡交换至第 i 行, 若找不到 return -1

代码:

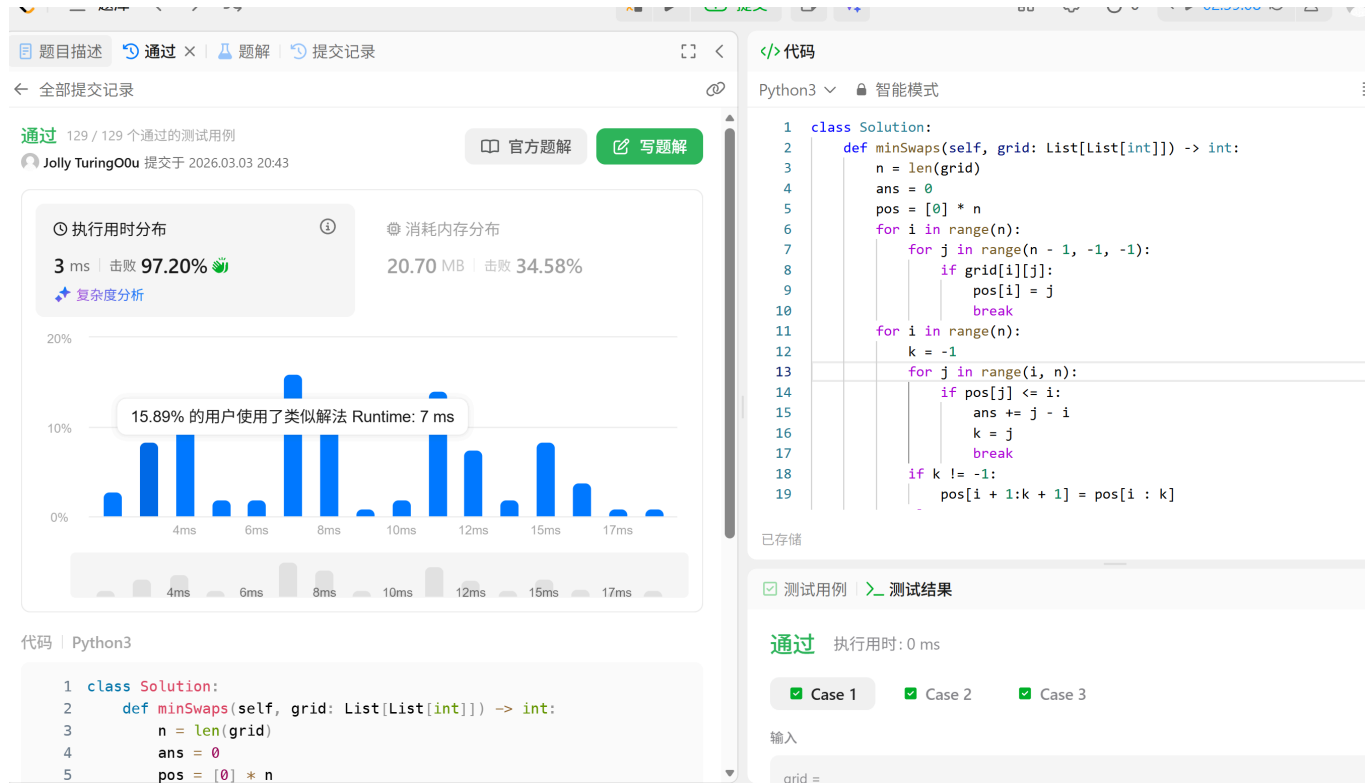
```
def minSwaps(self, grid: List[List[int]]) -> int:
    n = len(grid)
    ans = 0
    pos = [0] * n
    for i in range(n):
        for j in range(n - 1, -1, -1):
            if grid[i][j]:
                pos[i] = j
                break
    for i in range(n):
        k = -1
        for j in range(i, n):
            if pos[j] <= i:
                ans += j - i
                k = j
                break
        if k != -1:
            pos[i + 1:k + 1] = pos[i: k]
```

```

else:
    return -1
return ans

```

代码运行截图（至少包含有"Accepted"）



T20052:最大点数（同2048规则）

dfs, matrices, <http://cs101.openjudge.cn/pctbook/T20052/>

思路:

按照规则实现四个方向的移动，通过深度优先搜索枚举每一步的可能操作，计算全局最大。

代码:

```

def slide_and_merge(line):
    """
    核心合并算法：处理单行或单列的向左滑动和合并
    例如: [2, 0, 2, 4] -> [4, 4, 0, 0]
    """
    # 1. 移除所有 0: [2, 0, 2, 4] -> [2, 2, 4]
    len_line = len(line)
    new_line = [num for num in line if num != 0]
    merged_line = []
    skip = False
    for i in range(len(new_line)):
        if skip:
            skip = False
        else:
            if i < len(new_line) - 1 and new_line[i] == new_line[i + 1]:
                merged_line.append(new_line[i] * 2)
                skip = True
            else:
                merged_line.append(new_line[i])
    # 补0
    merged_line += [0] * (len_line - len(merged_line))
    return merged_line

```



```

        continue # 2. 检查相邻且相同的数字进行合并
    if i + 1 < len(new_line) and new_line[i] == new_line[i + 1]:
        merged_line.append(new_line[i] * 2)
        skip = True
    else:
        merged_line.append(new_line[i])
# 3. 在末尾补齐 0    merged_line += [0] * (len_line - len(merged_line))
return merged_line

```

通过矩阵变换实现四个方向的移动

```

def move_left(board):
    return [slide_and_merge(row) for row in board]

```

```

def reverse(board):
    """水平翻转矩阵"""
    return [list(reversed(row)) for row in board]

```

```

def transpose(board):
    """行列转置矩阵"""
    return [list(row) for row in zip(*board)]

```

```

def move_right(board):
    # 右移 = 先翻转 + 左移 + 翻转回去
    return reverse(move_left(reverse(board)))

```

```

def move_up(board):
    # 上移 = 转置 + 左移 + 转置回去
    return transpose(move_left(transpose(board)))

```

```

def move_down(board):
    # 下移 = 转置 + 右移 + 转置回去
    return transpose(move_right(transpose(board)))

```

```

def can_move(board):
    """检查棋盘是否还能进行有效移动"""
    for row in board:
        if 0 in row: return True # 有空格
    for i in range(m):
        for j in range(n):

```

```

        # 检查水平和垂直方向是否有相邻相同的数字
        if j + 1 < n and board[i][j] == board[i][j + 1]: return True
        if i + 1 < m and board[i][j] == board[i + 1][j]: return True
    return False

def dfs(step, board):
    if step == p or not can_move(board):
        global ans
        ans = max(ans, max([max(row) for row in board]))
        return
    old_board = [row[:] for row in board]
    board = move_left(old_board)
    dfs(step + 1, board)
    board = move_right(old_board)
    dfs(step + 1, board)
    board = move_up(old_board)
    dfs(step + 1, board)
    board = move_down(old_board)
    dfs(step + 1, board)
    board = old_board

board = []
m, n, p = map(int, input().split())
for _ in range(m):
    board.append(list(map(int, input().split())))
ans = 0
dfs(0, board)
print(ans)

```

代码运行截图 (至少包含有"Accepted")

#51862349提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```

def slide_and_merge(line):
    """
    核心合并算法: 处理单行或单列的向左滑动和合并
    例如: [2, 0, 2, 4] -> [4, 4, 0, 0]
    """
    # 1. 移除所有 0: [2, 0, 2, 4] -> [2, 2, 4]
    len_line = len(line)
    new_line = [num for num in line if num != 0]
    merged_line = []
    skip = False
    for i in range(len(new_line)):
        if skip:
            skip = False
            continue
        # 2. 检查相邻且相同的数字进行合并
        if i + 1 < len(new_line) and new_line[i] == new_line[i + 1]:
            merged_line.append(new_line[i] * 2)

```

基本信息

#: 51862349
 题目: T20052
 提交人: 25n2500011619
 内存: 3788kB
 时间: 200ms
 语言: Python3
 提交时间: 2026-03-04 21:40:11

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

收获：第一周的学习最大的收获是OOP方面的学习，通过Fraction类的学习，对于“封装”数据类型的便捷和应用方法都有了更多了解，特别是重新定义 `__add__`（‘+’号），`__str__` 和 `__lt__`（小于）方法，使代码更加标准化。另外，也熟悉了二进制的表示和位运算的函数，比如 `bit_count`，`bit_length`，位移，与或等等。2048棋盘的实现方法有一些复杂，我借鉴了老师提供的代码方法，其中通过`reverse`翻转、`transpose`转置来实现其他方向的方法实在没想到，这样只需实现一个方向的移动，其他只需流水线式的操作（也有一点封装操作的感觉）。